

Article

S-ZAC: Hardening Access Control of Service Mesh Using Intel SGX for Zero Trust in Cloud

Changhee Han , Taehun Kim , Woomin Lee  and Youngjoo Shin * 

School of Cybersecurity, Korea University, Seoul 02841, Republic of Korea; chhan22@korea.ac.kr (C.H.); taehunk@korea.ac.kr (T.K.); redcokeb@korea.ac.kr (W.L.)

* Correspondence: syoungjoo@korea.ac.kr

Abstract: In cloud services, the zero-trust security paradigm has emerged as a key strategy to reduce the large attack surface created by the complexity of cloud systems. Service mesh is a popular practice to realize the zero-trust architecture, which relies heavily on network access control to achieve the desired security. Building a service mesh-based solution in the cloud is not straightforward because privileged adversaries (e.g., malicious cloud insiders) can easily compromise the control plane where the access control function is implemented. In this paper, we propose S-ZAC, an access control hardening technique for service mesh-based solutions in the cloud. S-ZAC uses Intel SGX to provide a trusted execution environment for the control plane, which is responsible for enforcing access control for the service mesh. By isolating all access-control-related functions within an SGX enclave, S-ZAC ensures high resilience of the service mesh solution even in the presence of privileged adversaries. Due to the design limitations of SGX, implementing S-ZAC in the cloud for zero trust faces several challenges that can lead to serious scalability and failover issues. The first challenge is to establish secure communication channels between the S-ZAC components, even in the presence of privileged attackers. The second challenge is the limited memory capacity of the SGX enclave. Finally, the third challenge is that the inherent design of SGX does not support persistent enclave states, meaning that any state of running enclaves is volatile. We address these challenges by proposing our novel solutions. By implementing a prototype of S-ZAC, we evaluate its performance in terms of security and performance. The evaluation results validate the effectiveness of S-ZAC to enhance the security of the service mesh control plane in cloud environments.



Citation: Han, C.; Kim, T.; Lee, W.; Shin, Y. S-ZAC: Hardening Access Control of Service Mesh Using Intel SGX for Zero Trust in Cloud. *Electronics* **2024**, *13*, 3213. <https://doi.org/10.3390/electronics13163213>

Academic Editor: Javid Taheri

Received: 16 July 2024

Revised: 9 August 2024

Accepted: 12 August 2024

Published: 14 August 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: cloud computing; zero trust; service mesh; access control; Intel SGX

1. Introduction

The high level of complexity in cloud infrastructure makes it difficult for system administrators to detect and mitigate security threats from adversaries inside the cloud. *Zero trust* is a promising paradigm for ensuring the desired security of networked systems even in the presence of malicious insiders. In the zero-trust principle, as outlined in a National Institute of Standards and Technology (NIST) standard [1], every request for system resources must be authenticated and authorized prior to access.

Among the various practices for implementing zero trust, a service mesh-based framework is the most common approach due to its simplicity and popularity [2]. The service mesh approach uses a sidecar proxy for each workload, and the workload's networking tasks are delegated to the proxy. A sidecar proxy is an essential element in building zero trust because it performs authentication and authorization on each workload's request for system resources according to access control policies [3–5].

In the service mesh architecture, a control plane plays a critical role in access control. The main access control tasks in the control plane include (1) access control policy deployment and (2) policy validation. The policy deployment task is to distribute access control policies created by a system administrator to the access control solution across the

network. Policy validation is for the policy engine to verify whether or not requests to system resources are allowed according to the policy.

Its role in access control implies that the control plane must be protected with a high level of security; otherwise, adversaries could easily compromise the entire system and gain access to all unauthorized system resources by bypassing access control. However, in cloud environments, building the service network that satisfies the desired level of security is not straightforward due to the inherent characteristics of the cloud system. For example, Linux containers are vulnerable to container escape attacks (e.g., CVE-2019-5736 [6], CVE-2020-1527 [7]), which allow attackers to gain host privileges. The attack can easily compromise the control plane of the service network, which is isolated within a container.

Another characteristic of the cloud that makes it difficult to ensure control plane security is that cloud insiders (e.g., employees of the cloud provider) are not trusted. In general, (malicious) cloud insiders are assumed to have physical access to the servers hosting customer workloads [8–10]. This also becomes a major threat to the security of service mesh-based solutions built on top of the cloud infrastructure.

In this paper, we propose S-ZAC, an access control hardening technique for service mesh-based solutions in the cloud. S-ZAC leverages Intel Software Guard Extensions (SGX) technology [11] to create a trusted execution environment within the control plane of the service mesh. By confining all access control functions to an SGX enclave, S-ZAC achieves robust protection for the service mesh, maintaining its integrity even when faced with adversaries with elevated privileges. Specifically, S-ZAC processes that perform access control tasks are isolated within SGX enclaves. Therefore, even adversaries with host OS-level privileges are prohibited from accessing or manipulating service mesh control plane code and data, including access control policies that persist in process memory.

However, realizing S-ZAC is not trivial because of several challenging problems. The first challenge is to establish secure communication channels between the S-ZAC components. The naive way to achieve this is to use a TLS (Translation Layer Security) protocol.

However, in the presence of privileged attackers, such as a malicious cloud service provider (CSP), TLS certificates and private keys are likely to be leaked to the attackers [12]. The second challenge is the limited memory size of the SGX enclave. SGX typically supports an EPC size of less than 256 MB [13,14]. This limitation causes S-ZAC to support only a small number of workloads and a limited number of policies, limiting its scalability in the cloud. Finally, SGX does not support persistent enclave states by design, meaning that any state of running enclaves is volatile [15,16]. This prevents S-ZAC components from recovering their states after failures, which can occur for various reasons, such as software bugs in the components. We overcome these challenging problems by using our novel solutions, which are detailed in Section 5.

To evaluate the feasibility of S-ZAC, we implement a prototype and analyze its security and performance. Specifically, we use Gramine [17], a framework for SGX enclave applications, to implement the control plane of the service mesh within the SGX enclave as well as all the functionalities of SGX remote attestation. Among the various remote attestation APIs supported by Gramine, we use Intel SGX Data Center Attestation Primitives (DCAP) [18] because it runs standalone and does not rely on any trusted third party like Intel Trust Authority [19]. To analyze the effectiveness of S-ZAC, we compare the performance between the S-ZAC process without SGX and an existing service mesh solution.

The contributions of this paper are as follows:

1. We propose S-ZAC, an access control hardening technique that utilizes Intel SGX, to enhance the trustworthiness of the service mesh control plane for zero trust in the cloud.
2. We address several challenges for implementing S-ZAC, which may reduce its feasibility, and propose novel solutions to overcome these problems.
3. We implement a prototype of S-ZAC by utilizing Gramine, an SGX-based application development framework. Our prototype also implements SGX's remote attestation to guarantee the reliability of the control plane against network adversaries.

The organization of this paper is as follows: Section 2 presents related work, and Section 3 provides background knowledge. Section 4 outlines the threat model and design goal. Section 5 explores the design of the S-ZAC architecture, and Section 6 describes the implementation of S-ZAC. The performance evaluation results are presented in Section 7. Section 8 provides the security analysis and discussion for S-ZAC. The paper concludes with Section 9.

2. Related Work

2.1. Security Solutions for Service Mesh

There are some studies that focus on improving the security of service mesh frameworks for the zero-trust architecture. Adam et al. introduced a security-enhancing technique using Intel SGX within the service mesh [20]. It appears similar to our work but has significant differences. Specifically, they suggest isolating the data plane workloads of the service mesh within an SGX enclave, while our work focuses on the isolation of the control plane. Although they also introduced security hardening techniques for the control plane using a shared signature scheme, the control plane is subject to insider attacks by malicious CSPs, which can be mitigated by S-ZAC. More specifically, S-ZAC aims to enhance the security of the service mesh's control plane by ensuring confidentiality, integrity, and availability, even in the presence of privileged adversaries. In contrast, Adam et al. only address the integrity of the control plane in the service mesh. This limited security approach makes their framework vulnerable to attacks such as leaking access control policies from memory, terminating processes, and corrupting executable files.

Zhang et al. proposed a method to enhance the security of SDP (Software Defined Perimeter), one of the promising service mesh-based implementations for zero trust [21]. Their approach is to utilize eBPF (extended Berkeley Packet Filter); an eBPF program running on the control plane performs embedding authentication data into every valid SDP packet and filters out any unauthorized packets. In addition, Sedghpour et al. argue for the use of eBPF in combination with service mesh as a way to not only improve the security of service mesh but also to increase visibility and reduce the complexity of operating the solution [4]. They also show how to adopt eBPF by comparing an actual service mesh solution Cilium [22] implemented using eBPF to other solution architectures. Furthermore, Duong et al. propose a 5G network-based service mesh that uses Cilium to improve security and visibility of the control plane [23].

Hussain et al. proposed an approach that achieves strong isolation of the control plane of a service mesh by adopting API gateways [24]. Kang et al. proposed a service mesh protection for container-based network services where workload containers are equipped with a single NIC [25]. They proposed a solution that employs cryptographic algorithms to isolate control traffic, thereby addressing the issue of sensitive control traffic being unintentionally exposed to the data plane.

While these approaches address the security issues of the service mesh control plane, they are still vulnerable to attacks by strong adversaries with host OS-level privileges or physical access to the machine. Unlike previous solutions, our work builds a service mesh upon hardware-assisted trusted execution environments where all security-sensitive functions, including policy deployment and validation of the service mesh control plane, are located.

On the other hand, several studies have focused on access control hardening mechanisms without relying on the Intel SGX technology [26–28]. These works have enhanced the security of access control mechanisms using blockchain, smart contracts, and multi-authority. However, their works mainly focus on privacy-preserving sensitive data such as healthcare data, and they do not account for privileged adversaries (e.g., malicious cloud insiders) as a security threat. Table 1 presents a comparison to related works that propose security solutions for service mesh.

Table 1. A comparison of security solutions for service mesh.

Name	Protection Target	TEE	Malicious CSP	Technique
Zhang et al. [21]	Control and data plane	✗	✗	Introduce an eBPF-based Dynamic Perimeter
Sedghpour et al. [4]	Control and data plane	✗	✗	Combine service mesh with eBPF
Duong et al. [23]	Control plane	✗	✗	Integrate Istio and Cilium
Hussain et al. [24]	Control plane	✗	✗	Adopt API gateway
Kang et al. [25]	Data plane	✗	✗	Employ traffic separation and cryptographic algorithms
Adam et al. [20]	Data plane	✓	✓	Use hardware-assisted TEE
S-ZAC (Our work)	Control plane	✓	✓	Use hardware-assisted TEE

2.2. Security Enhancing Techniques Using TEE

There is lots of research addressing the security vulnerabilities of existing applications when faced with OS-level privileged adversaries. All of this work typically uses hardware-assisted trusted execution environments (TEEs), such as Intel SGX, to isolate security-sensitive layers of applications within the TEE. For example, there are several works that augment key-value stores [29], databases [30], and storage [31] with SGX enclaves to ensure confidentiality and integrity of stored data under the compromised OS. In addition, other works aim to address the security weaknesses of network applications such as content-based routing engines [32] or DNS [33] and even existing security applications like firewalls [34], IDS [35] and Tor’s ecosystem [36] by employing Intel SGX.

Although seemingly similar, our work differs from the previous work in two ways. First, S-ZAC is the first solution to strengthen the security of the control plane of a service mesh by leveraging Intel SGX technology. Second, while the previous work mainly focused on strengthening the security of specific components or layers of applications, our work addresses the security of the entire cloud by protecting the control plane of microservice architecture. In this way, S-ZAC aims to establish a secure zero-trust architecture for the cloud. Table 2 shows a comparison to related works that focus on security-enhancing techniques using TEE.

Table 2. A comparison of security enhancing techniques using Intel SGX technology.

Name	Application	Category
Precursor [29]	Key-value stores	Data management
EnclaveDB [30]	Database	Data management
Yang et al. [31]	Storage	Data management
Pires et al. [32]	Routing engines	Networking
PDOT [33]	DNS	Networking
SENG [34]	Firewall	Security application
Nakano et al. [35]	IDS	Security application
Kim et al. [36]	Tor’s ecosystem	Security application
S-ZAC (Our work)	Service mesh	Cloud

3. Background

3.1. Zero-Trust Architecture

Zero trust is a new security paradigm that considers the possibility of attackers infiltrating internal systems. It recommends authentication and authorization for all workloads within the system. With the increase in security incidents within internal systems, the importance of zero trust has been further emphasized. As a response to this, NIST has provided guidance on implementing an architecture for zero trust to promote the creation of secure systems [1].

Zero-trust architecture proposed by NIST is as follows: When a user requests access to internal system resources, a gateway receives the request and forwards it to the policy engine within the control plane for access control policy validation. This approach to access control is implemented for all resources within the system, strengthening the security of resources and enhancing overall security to guard against intrusions by potential attackers within the system.

3.2. Service Mesh

A service mesh functions as an infrastructure layer solution for applications structured on a micro-services architecture [37]. It involves deploying a sidecar proxy on each workload within the micro-services architecture, responsible for managing all network operations within that workload. Under the control plane's supervision, network tasks throughout the system can be effectively managed, offering additional functionalities such as network security features like monitoring, access control, and anomaly detection.

The service mesh control plane consists of a manager and an agent, as shown in Figure 1. The manager, located at a master node, is responsible for service management, network monitoring, and policy administration, while the agent, located at a worker node, performs health checking, network status reporting, and access control policy enforcement. Since this paper focuses only on the access control function, we provide more detail on the relevant features of the control plane.

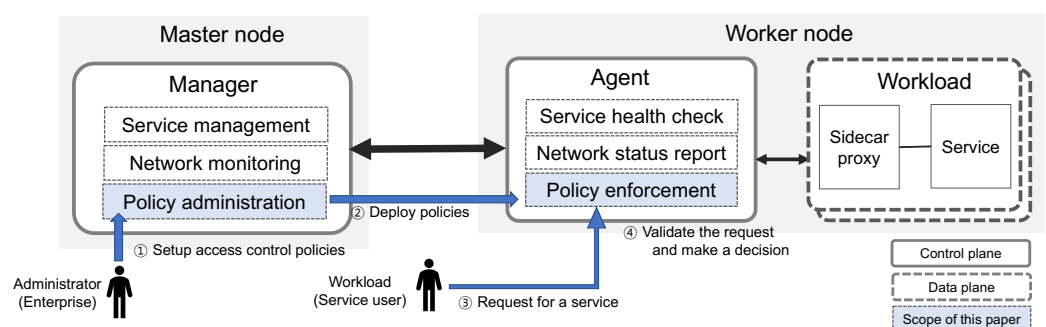


Figure 1. Service mesh framework.

Regarding policy administration, the manager provides administrators with an interface to configure access control policies for each workload (Step ① in Figure 1). The configured access control policies on the manager will be deployed to remote agents over the network (Step ②). The policy enforcement module in the agent is then able to authorize each request to a service in the workload based on the access control policies. For instance, a service user (i.e., other workloads in the service mesh) who initiates a session with the workload first sends an authorization request for the service to the agent (Step ③). The agent then validates the request by matching it with the access control policies and decides whether to accept the request or not based on the validation result (Step ④).

This access control process aligns with the recommended zero-trust access control system. Every resource within the micro-services architecture is shielded by the sidecar proxy, ensuring that all network communications are authenticated and access-controlled. Recognizing these advantages, NIST has released a guide document endorsing the use of service mesh for securing micro-services architecture [38].

3.3. Intel SGX

To mitigate potential threats like cold boot attacks [39] posed by adversaries with physical access or operating system-level privileges, Intel introduced SGX technology. SGX creates a trusted execution environment for processes. All trusted code and data are loaded and executed within an isolated runtime context called an SGX enclave. As the SGX technology places the root of trust in the processor, it ensures the confidentiality and integrity of the code and data running within the enclave.

Memory encryption. To ensure the confidentiality and integrity of applications running inside the enclave, Intel has implemented a memory encryption engine (MEE) [40] in its processors. The MEE encrypts the memory regions that enclaves use with a 128-bit key. Furthermore, the MEE offers integrity for enclaves by introducing an integrity tree. The integrity tree is constructed from a modified Merkle tree, utilizing a stateful message authentication code (MAC) algorithm. The statefulness of the MAC algorithm prevents

adversaries from modifying the entire integrity tree, which ensures the strong memory integrity of the enclave.

SGX attestation. The attestation feature within SGX is a crucial security component designed to guarantee the integrity and authenticity of the software executing within an enclave. This attestation comes in two forms: local attestation, which is applicable within the same host, and remote attestation, which relates to an enclave operating on a remote host. While local attestation can be established without trust authority, remote attestation requires a third party to verify an enclave located at a remote host. Intel offers two distinct remote attestation mechanisms: EPID (enhanced privacy ID [41]) and DCAP (data center attestation primitives [18]). Each type depends on the type of required third party. In this paper, we choose the DCAP mechanism to implement remote attestation within a cloud cluster consisting of private networks.

RA-TLS. RA-TLS (Remote Attestation-TLS) [42] is a prime example of a TLS protocol that uses SGX remote attestation to improve resource efficiency and communication security. Specifically, it leverages SGX remote attestation for a TLS handshake protocol. Remote peers authenticate each other with an extended TLS certificate that additionally contains SGX quotes used for attestation. Because persistent private keys are stored within the SGX enclave and never exposed to the outside world, private keys can be kept secret.

An enclave wishing to use RA-TLS must initiate a RA-TLS provisioning procedure. Through this process, the enclave obtains an SGX quote value from a trusted third party provided by Intel, where the quote is issued and signed by the third party's private key. It then generates a public/private key pair and an X.509 certificate. The quote value is included in the extension field of the certificate to perform remote attestation.

The issued quote value and the extended X.509 certificate are used for the TLS handshake process in RA-TLS. This enables stronger authentication operations during the handshake process and keeps the private keys secure within the enclave. In this work, we utilize RA-TLS as the core technology for securing network traffic and authenticating the control plane.

4. Problem Formulation

4.1. System Model

Our study focuses on a cloud service architecture that enforces zero trust based on a service mesh framework, as shown in Figure 2. Specifically, we study a cloud system that includes a cloud service provider (CSP), an enterprise, and the service user.

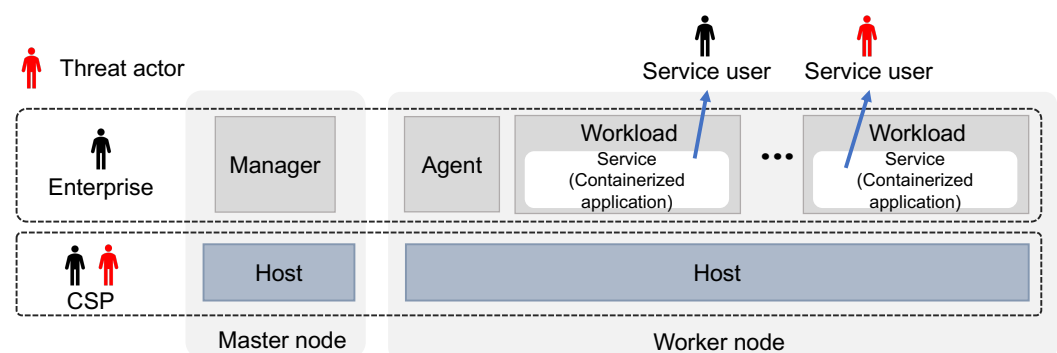


Figure 2. System and threat model.

CSP. It provides computing resources to customers (i.e., enterprises) through an Infrastructure as a Service (IaaS) service delivery model. Computing resources include physical or virtualized host servers, allowing customers to build their own service mesh framework. The CSP maintains access rights to the infrastructure it provides, enabling it to monitor usage while its solutions are in use.

Enterprise. It is a customer of CSPs. That is, it builds the enterprise-specific service utilizing computing resources provided by CSPs on the basis of the service mesh archi-

texture. The service is delivered to the service users (e.g., customers of the enterprise or employees). The enterprise service can be built in the form of containerized applications. Through the service mesh's control plane, access to any resources offered by the service is only granted to the service users who have the appropriate permissions.

Service user. It is an entity that uses services provided by the enterprise. The service users are confined to their own workspaces, commonly provided as an individual service in a containerized application. Access to the workspaces of other service users without permission is strictly restricted.

4.2. Threat Model

4.2.1. Threat Actors

In our threat model, we consider two types of attackers: one playing the role of the service user and the other playing the role of a CSP (see Figure 2). Details of the threats posed by each attacker are outlined below.

Malicious service user. This type of attacker has legitimate access to their own containerized workspace, which is isolated from the host environment. The malicious service user may attempt to escape the containerized application and gain root privileges on the host. To do this, he/she may exploit some vulnerabilities in Linux containers (e.g., runc in Docker). Once root privileges are obtained, the attacker will attempt to access other workloads running on the same worker node.

Malicious CSP. The malicious CSP has all the permissions to access the underlying resources of the hardware infrastructure used by the enterprise. That is, the attacker has access to both worker node hosts, including root privileges, and master node host access rights. Furthermore, he/she can obtain the API keys and secret values of the system established by the enterprise. As a result, the attacker can attempt unauthorized access to their desired workloads by bypassing the enterprise's access control using legitimate permissions.

4.2.2. Threats

We assume that the attackers considered above have two attack goals: (1) circumventing access control checks and (2) disturbing access control enforcement, which become actual security threats to the control plane for access control.

T1: Circumventing access control checks. One of the security threats posed by an attacker targeting a control plane is to bypass access control checks to gain unauthorized access to prohibited workload resources. The attacker may attempt to bypass the access control checks using several attack techniques, such as (A1) leaking access control policies, (A2) manipulating access control policies in the memory, and (A3) stealing control plane secrets.

T2: Disturbing access control enforcement. Another security threat can arise from attackers with the intent of disturbing access control operations. This will harm the availability of the service mesh control plane. The attacker may attempt to accomplish this goal using several attack techniques, such as (A4) terminating process, (A5) corrupting executable files, (A6) man-in-the-middle (MITM) attack between an access control agent and a workload, and (A7) MITM attack at the master.

4.2.3. Other Assumptions

In this paper, Intel SGX enclave vulnerabilities such as Denial-of-Service (DoS) [43], side-channel attacks [44–46], and other physical attacks [47,48] on the CPU are considered out of scope.

4.3. Security Goal

S-ZAC aims to improve the security of the service mesh's control plane, particularly in response to the threats outlined in Section 4.2. More specifically, we present two concrete security goals of S-ZAC as follows:

- (Security goal 1) *Defending against a threat T1*. To mitigate a security threat, circumventing access control checks, S-ZAC must ensure confidentiality and integrity in the control plane. That is, S-ZAC must protect the control plane from attackers who attempt to directly access and manipulate a memory of host processes that hold confidential data such as control plane secrets or access control policies.
- (Security goal 2) *Defending against a threat T2*. To mitigate threat T2, S-ZAC must guarantee the availability of the control plane. In particular, S-ZAC must protect the control plane from attackers attempting to maliciously control host processes or manipulate executable files to disrupt access control operations.

5. Design

The design of S-ZAC follows the general architecture of a service mesh consisting of a master node and multiple worker nodes. Figure 3 illustrates the overall architecture of S-ZAC.

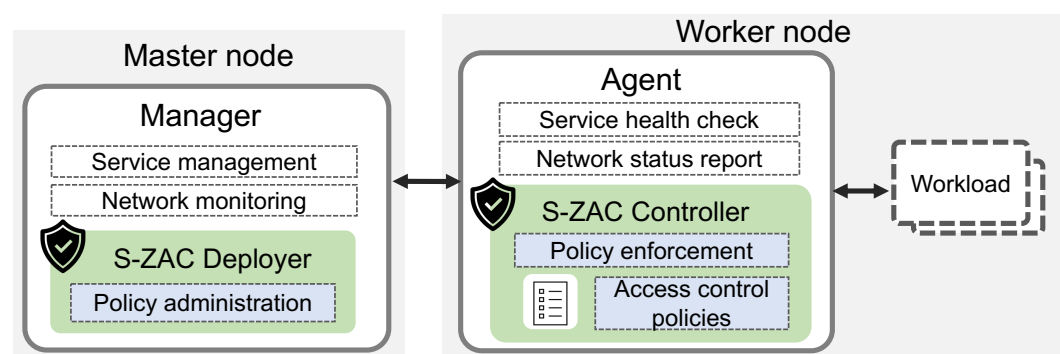


Figure 3. S-ZAC architecture.

5.1. Overview

In a master node, S-ZAC has a *Deployer* that is responsible for policy management and deployment. It provides a user interface for an enterprise administrator to create and manage access control policies for each worker node. The deployer then transfers the access control policies created by the administrator to the controllers in the worker node. The deployer maintains a configuration of all workloads running in the cluster for the purpose of distributing access control policies. Cluster information is periodically updated through administration and management tools.

In each worker node, S-ZAC has a *Controller* responsible for enforcing access control based on the policies deployed by the master node. For the enforcement of access control, it maintains a list of access control policies, delivered by the deployer, in the enclave. Upon receipt of an access request from other workloads, it performs validation of the request against the policies. The decision to allow or deny the access request is made based on the validation result. All of the executable code for access control enforcement and the relevant policies, including access control policies, are isolated within the controller's enclave. This design prevents privileged attackers from compromising the process responsible for access control enforcement, thus ensuring the confidentiality and integrity of access control policies in the memory.

5.2. Challenging Problems and Solutions

Implementing S-ZAC for zero trust in the cloud is not trivial due to several challenges. We describe the challenges in detail, and propose novel solutions to address them.

Establishing secure channels. While the control plane processes are isolated within the SGX enclave, the network communication between them must be protected from attackers. The naive way to implement a secure network channel is to use TLS in S-ZAC. However, in the presence of privileged attackers (e.g., a malicious CSP), TLS certificates and private keys are likely to be leaked to the attackers. We address this issue by using

RA-TLS. In RA-TLS, all secret key derivation processes are combined with SGX remote attestation, and all derived secrets, including an X.509 private key, are kept within an SGX enclave. Because these secrets remain within the enclave throughout their lifecycle, they can be protected from privileged attackers. Certificates generated using these internally generated private keys are self-signed, and trust in the certificates is verified using SGX quote values and reporting data contained in the certificate.

S-ZAC uses DCAP (Data Center Attestation Primitive) for the remote attestation because it allows third parties to have their own remote attestation infrastructure in internal data centers.

Limited SGX enclave memory. The Intel SGX enclave typically supports a limited EPC size of less than 256 MB. This limitation results in S-ZAC supporting only a small number of workloads and a limited number of policies, limiting its scalability in the cloud.

We overcome the SGX memory limitation by proposing an efficient data structure for access control policies. In our data structure, an individual policy consists of two parts, *index* and *rule*, as shown in Figure 4. The index represents the label of the workload that serves as an identifier for applying the policy, while the rule contains information about the workload that is required to determine whether it should be allowed or denied. It is the rule part that significantly contributes to the overall size of access control policies, while the size of the label part is confined to the number of workloads.

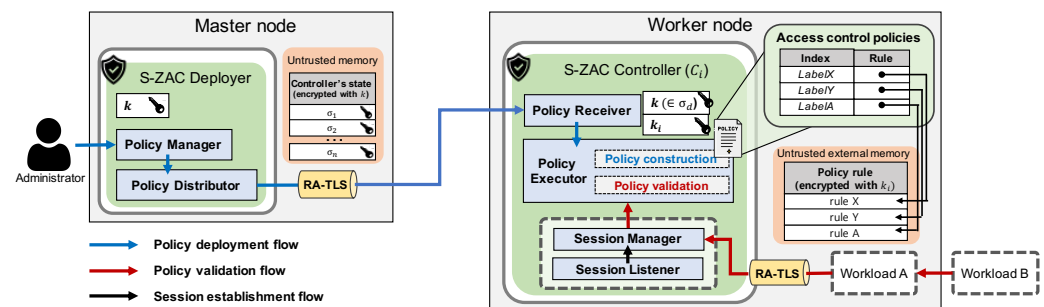


Figure 4. S-ZAC implementation.

Our approach is that we only keep the label part within the SGX enclave memory, while the rule part is exported to the main memory outside the enclave. The index part contains pointers that link to the location of relevant rule data in the main memory, allowing for fast lookups when validating access requests with the policies. To maintain the confidentiality of the rule data, S-ZAC performs encryption on the data before exporting it to untrusted memory.

Non-persistent state of SGX enclave. S-ZAC components can fail for various reasons, such as software bugs in the components, resulting in SGX enclave restarts. However, by design, SGX technology does not support the persistent state of the SGX enclave, which means that any state information of the running enclave is volatile. Although it is complemented by SGX sealing technology [49], which provides persistent storage for enclaves, serious security vulnerabilities have been discovered in SGX sealing [50]. Using SGX sealing exposes S-ZAC to new attack surfaces.

In light of this, we have to devise a method to persistently store the state of S-ZAC components within the enclave without using SGX sealing. Our solution is to share volatile secret states across S-ZAC components so that the state of a failed component can be recovered using other components that hold the state. More specifically, the deployer holds all the states $\sigma_i (1 \leq i \leq n)$ of n controllers in its SGX enclave memory. Upon a restart of a controller C_i due to a failure, its state σ_i can be recovered using the deployer. The state σ_i contains an ephemeral secret key that is used to encrypt internal data, such as C_i 's access control policies. States are exchanged over a secure channel established between a controller and a deployer.

Like this, the persistence of deployer's state, σ_d , can also be ensured by distributing it to $m(\leq n)$ controllers over secure channels. Since the deployer holds all the controller's states, σ_d also contains the information k necessary to recover those states if the deployer fails. For efficiency, we choose to store the controller's states in the deployer's persistent storage in encrypted form. The information k is used as a secret key to encrypt the state information.

6. Implementation

We implement S-ZAC based on the design presented in the previous section. For the implementation, we use Gramine [17], a lightweight library operating system that is optimized to execute a whole application with minimal dependencies on the host system. Gramine enables running applications in a secure and isolated environment, such as SGX enclaves. Each S-ZAC component is encapsulated by Gramine so that working threads for the control plane are executed within enclaves. Figure 4 shows the internal structure of our implementation.

S-ZAC works in two phases: an *initialization phase* and an *operational phase*, with an additional phase for *recovery for failure*; each phase has specific operations associated with it. The procedures for each S-ZAC operation are detailed in the following subsections.

6.1. Initialization Phase

In this phase, the S-ZAC components are initialized prior to operation.

Initializing Deployer. Once placed on a master node, the deployer gets initialized. Algorithm 1 presents the pseudocode utilized for the deployer initialization process, and its initialization procedures are as follows:

Algorithm 1: Pseudocode for Deployer initialization

```

1 procedure policy_distributor():
2   | init_RA-TLS_provisioning()
3 end
4
5 procedure policy_manager():
6   | wait_access_control_policy()
7 end
8
9 thread_create(policy_manager)
10 thread_create(policy_distributor)
11  $k \leftarrow \text{generate\_secret\_key}()$ 
12 distribute_state( $\sigma_d$ )

```

- Step 1. Gramine loader creates an enclave runtime context and invokes two main deployer threads, a *policy manager* and a *policy distributor*, within the process context.
- Step 2. The deployer generates a secret key k , which is used to encrypt σ_i (i.e., controller C_i 's state information) in the storage, where $1 \leq i \leq n$, and n denotes the number of controllers. k comprises the deployer's state σ_d . After initializing controllers, the deployer distributes σ_d to $m(m \leq n)$ controllers.
- Step 3. When invoked, the policy distributor initiates an RA-TLS provisioning procedure that consults Provisioning Certificate Caching Service (PCCS) to obtain an SGX quote value for remote attestation. It uses the obtained quote to generate a public and private key pair and a TLS certificate.
- Step 4. When invoked, the policy manager begins waiting for an administrator to enter access control policies via a user interface.

Initializing Controller. After the deployer initialization, the controller C_i is placed on each worker node. Algorithm 2 shows the pseudocode for the controller initialization procedure, and its initialization process is described as follows.

Algorithm 2: Pseudocode for Controller initialization

```

1 procedure policy_receiver():
2   | init_RA-TLS_provisioning()
3   | establish_RA-TLB_with_policy_distributor()
4 end
5
6 procedure policy_executor():
7   | wait_policy_execution()
8 end
9
10 procedure session_manager():
11   | wait_session_managing_requests()
12 end
13
14 procedure session_listener():
15   | init_RA-TLS_provisioning()
16   | wait_session_requests()
17 end
18
19 thread_create(policy_receiver)
20 thread_create(policy_executor)
21 thread_create(session_manager)
22 thread_create(session_listener)
23  $k_i \leftarrow \text{generate\_secret\_key}()$ 
24  $\sigma_d \leftarrow \text{exchange\_state\_with\_Deployer}(\sigma_i)$ 

```

- Step 1. Gramine loader invokes four main controller threads, *policy receiver*, *policy executor*, *session manager*, and *session listener*, within an enclave's runtime context.
- Step 2. The controller C_i generates a secret key k_i , which is used to encrypt data located in the external memory, such as C_i 's access control policies. k_i comprises the C_i 's state information, σ_i .
- Step 3. When invoked, the policy receiver and the session listener individually initiate an RA-TLS provisioning to obtain SGX quote values, in the same way as Step 2 of the deployer initialization.
The policy receiver establishes an RA-TLS session with the policy distributor. The session is used for receiving policy information from the deployer.
- Step 4. Using the master secret shared between the deployer and controller C_i on the TLS handshake, the deployer and controller derive secret k_i , which comprises the state σ_i . After that, the deployer encrypts σ_i using its secret key k and stores the encrypted data in external memory. It then sends σ_d to C_i over the secure channel.
- Step 5. The session listener begins waiting for session requests from workloads that want to make a new connection with the Controller.

6.2. Operational Phase

In this phase, S-ZAC works with three operations: policy deployment, workload initialization, and policy validation.

Policy deployment. To deploy policies for a workload, an administrator transmits a policy file to the policy manager at the deployer. Figure 5 illustrates the policy deployment process as a sequence diagram, and the policy deployment process between the deployer and the controller is as follows:

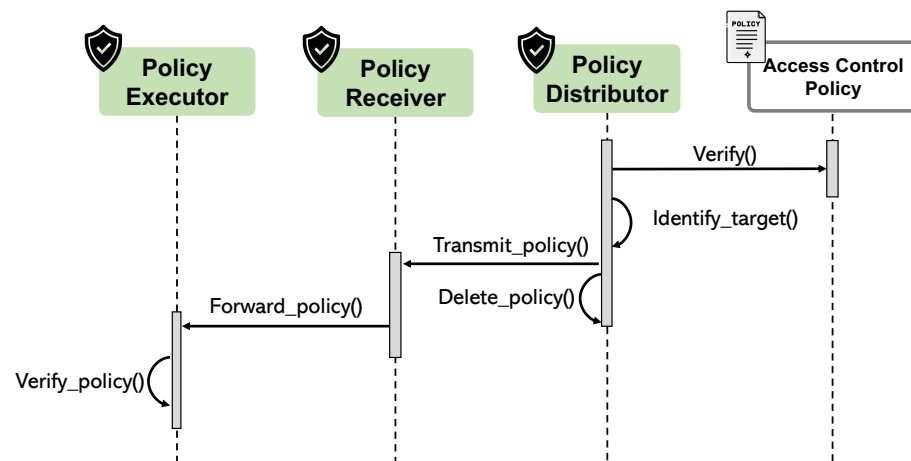


Figure 5. Sequence diagram for policy deployment process.

- Step 1. The policy distributor in the deployer verifies access control policies in the policy file and identifies the target worker node to deploy the policies.
- Step 2. The policy distributor transmits the policy data to the dedicated controller via the RA-TLS session. The policy distributor then immediately deletes the policies to ensure that no footprints of the policy data remain in its storage.
- Step 3. The policy receiver in the controller receives the policy data from the policy distributor and then forwards it to the policy executor.
- Step 4. The policy executor verifies the legitimacy of the policy data. The ‘rule’ field within the policy data is encrypted with k_i and is kept external to the enclave. On the other hand, the ‘index’ field resides within the enclave, containing the pointer to the ‘rule’.

Workload initialization. When a new workload is deployed on a worker node, proper initialization is required to associate its function with S-ZAC. Specifically, a new RA-TLS session should be established with the controller on the same worker node. The established session will be maintained throughout the lifecycle of the worker node. Figure 6 presents the workload initialization process as a sequence diagram. Steps for the workload initialization are as follows:

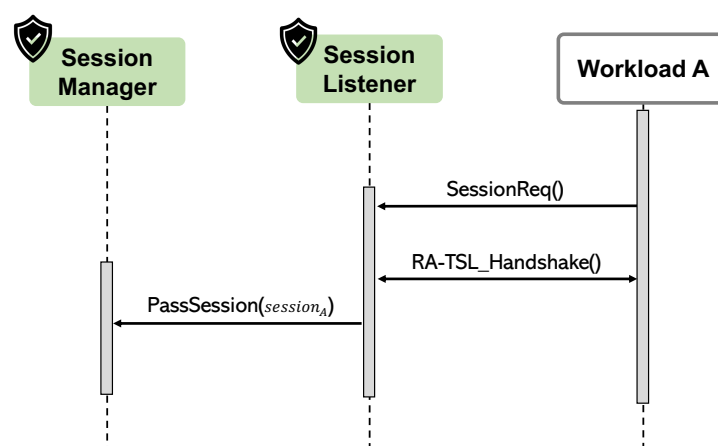


Figure 6. Sequence diagram for workload initialization process.

- Step 1. The newly deployed workload sends a new RA-TLS session request to the session listener in the controller:
- Step 2. The session listener initiates an RA-TLS handshake with the workload and establishes a session.

Step 3. The created session is passed to the session manager for further management. The session is maintained as long as the workload is alive in the node. When the session expires, it is removed by the session manager.

Policy validation. Once the controller receives access control requests from workloads, it validates the request based on the access control policies. The policy validation process is illustrated as a sequence diagram in Figure 7, and its operational procedures are as follows:

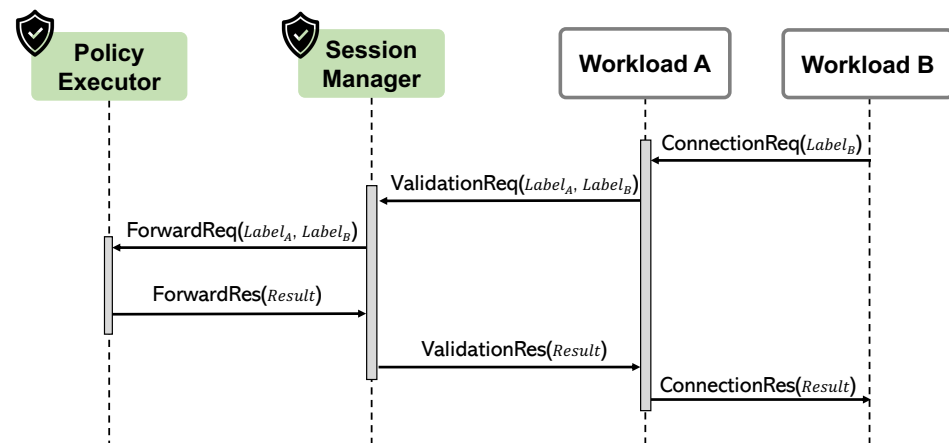


Figure 7. Sequence diagram for policy validation process.

- Step 1. Workload B, placed on another worker node (in Figure 4), sends a connection request to Workload A. The request includes relevant label values ($Label_B$).
- Step 2. Workload A then sends a validation request with labels $Label_A$ and $Label_B$ to the controller to check the validity of the request of Workload B. Note that all these communications are conducted during the RA-TLS session.
- Step 3. The session manager in the controller receives the validation request and forwards it to the policy executor.
- Step 4. The policy executor looks up the address of the 'rule' by matching the $Label_A$ on the map of 'index'. Using the address found, the policy executor loads the 'rule' data from untrusted memory and decrypts the data with k_i . The access control validation is performed using the loaded 'rule' data.
- Step 5. Once the validation is completed, the executor responds to the results back to Workload A through the session manager. Based on the results of the validation, Workload A decides whether to allow or deny the connection request from Workload B.

6.3. Recovery Phase

In this phase, S-ZAC recovers the state of a failed component. Figures 8 and 9 present the deployer's and controller's recovery phase from the failover as a flow chart, respectively.

Deployer recovery. If a deployer fails, it recovers its state through the following steps:

- Step 1. It re-initializes itself and initiates RA-TLS sessions with controllers.
- Step 2. It requests its state σ_d to controllers over the RA-TLS session.
- Step 3. Upon receipt of σ_d , it recovers the internal data (i.e., encrypted controllers' states $\sigma_1, \dots, \sigma_n$) with an encryption key k in σ_d .

Controller recovery. If a controller C_i fails, it recovers its state through the following steps:

- Step 1. It re-initializes itself and initiates RA-TLS provisioning. The policy receiver and the session listener recover the RA-TLS session with the deployer and every workload running on the same worker node. Then it recovers the list of workloads running on the same worker node.
- Step 2. It requests its state σ_i to the deployer.

Step 3. Upon receipt of σ_i , it recovers the encrypted internal data (e.g., a part of access control policies stored in untrusted external memory) with an encryption key k_i in σ_i .

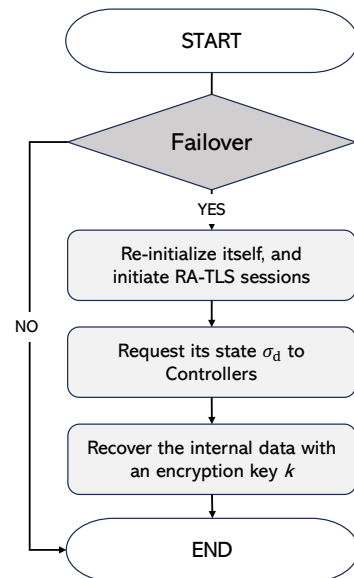


Figure 8. A flow chart for deployer recovery.

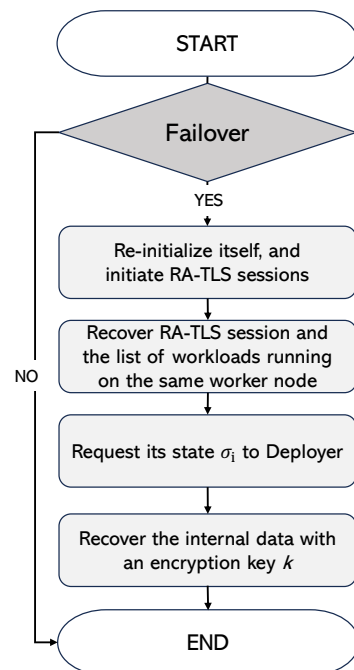


Figure 9. A flow chart for controller recovery.

6.4. Integration with Service Mesh Frameworks

We designed S-ZAC to be easily integrated with existing implementations of the service mesh framework. That is, modules for access control administration and enforcement in the current frameworks can be replaced by S-ZAC deployer and controller with little engineering effort to modify the sidecar proxy.

For instance, S-ZAC can be easily integrated with HashiCorp Consul [51], one of the most popular service mesh solutions. Consul operates a client agent on each worker node and manages workloads by offering a policy-based access control mechanism. To integrate

with S-ZAC, we simply alter the behavior of the Consul sidecar proxy so that it reroutes access control validation requests to S-ZAC instead of the original access control module.

Istio [52] is another popular service mesh framework that S-ZAC supports in its integration. Istio differs from Consul in that it performs access control enforcement through the sidecar proxy. Hence, we can integrate S-ZAC by adapting the proxy so that it forwards access control requests to S-ZAC's controller. This modification not only enhances Istio's access control functions but also potentially reduces resource consumption on the sidecar proxy, as Istio lacks a control plane component for policy validation.

These examples show the adaptability of S-ZAC in enhancing the access control mechanisms of various service mesh solutions, underscoring its versatility and efficiency in contemporary network security contexts.

7. Performance Evaluation

With our implementation presented in the previous section, we conducted several experiments to evaluate the overhead performance of S-ZAC in terms of access control enforcement and provisioning.

Experimental setup. Our experimental setup consists of three physical machines, one acting as a master node and two acting as worker nodes, which are connected together via a 300 Mbps Ethernet network. Each node is equipped with the Intel Core i7-8700 CPU and 8 GB of RAM, running Ubuntu 64-bit Linux 20.04 with a kernel version of 5.15.0-87-generic. The roles of each node are consistent across S-ZAC and other service mesh solutions, such as Istio and Consul.

7.1. Access Control Enforcement Overhead

The S-ZAC Controller can be a performance bottleneck because it is a single point that performs the validation of each access control request for all workloads in a worker node. Hence, we evaluate the performance overhead of the controller with regard to access control enforcement.

7.1.1. Overhead Due to a Large Number of Requests

We evaluate the performance overhead of the controller under the scenario where multiple workloads concurrently initiate access control requests to the controller. In the experiment, we measure response time and loss rates while varying the number of requests in comparison to other existing service mesh solutions. We also examine the performance impact of S-ZAC due to its usage of SGX enclaves in access control enforcement. For this purpose, we implemented a modified version of S-ZAC that does not use SGX enclaves and compared it with the original implementation.

Figures 10 and 11 show the experimental results. The results show that Istio's response time spikes when the number of requests per second exceeds 200, but the loss rate remains zero. We infer from the results that it is because Istio places a higher priority on reliability than on availability, focusing on serving as many requests as possible at the expense of a longer response time. On the other hand, Consul's response time increases steadily with the number of requests per second without any spikes, but its loss rate increases dramatically when the number of requests per second exceeds 200.

S-ZAC has a lower response time than Istio and a lower loss rate than Consul. Although the loss rate is low, it is negligible compared to the overall performance. By observing the point where the response time starts to increase, we conclude that S-ZAC has higher availability than Istio. We also observe that S-ZAC has a lower response time than Consul when Intel SGX is disabled. This difference is attributed to the inherent execution latency introduced by the use of SGX enclaves.

In terms of network performance, S-ZAC shows reasonable performance with a negligible request loss rate. S-ZAC outperforms Istio even with the increased response time due to the use of the SGX enclave.

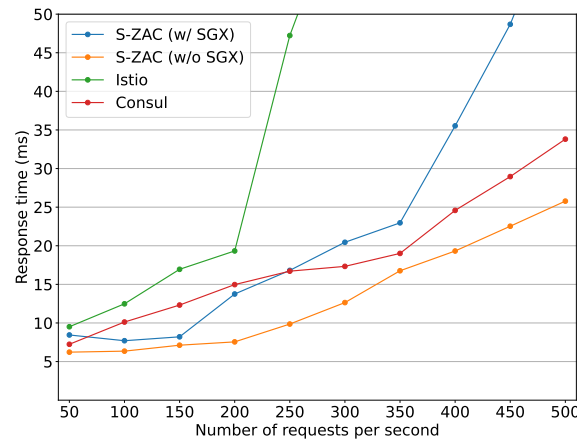


Figure 10. Response time with respect to the number of requests.

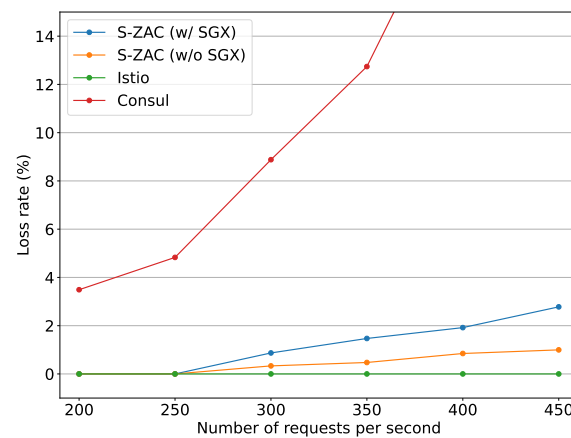


Figure 11. Loss rate with respect to the number of requests.

7.1.2. Overhead Due to a Large Number of Access Control Policies

Since the S-ZAC controller maintains a list of access control policies in an SGX enclave, each request validation is performed within the enclave. This can become another performance bottleneck due to the SGX runtime overhead. To evaluate the overhead, we measure the S-ZAC policy validation performance and compare it to the measurement result obtained in a case where no SGX enclave is used. Since performance is affected by the number of access control policies stored in memory, we also measure the validation execution time with respect to the number of policies. We measure how long it takes to perform a validation check by varying the number of access control policies from 1000 to 100,000 based on the policy 'rule'. We obtain the measurement results by averaging the execution times of 100,000 independent experiments.

The experimental results are shown in Figure 12. From the results, we observe a correlation between the number of access control policies and the execution time of the request validation. Specifically, validation takes about 26 microseconds for a dataset of 1000 policies in the case of S-ZAC with SGX enabled. This execution time increases to 30 microseconds for 10,000 policies and 40 microseconds for 100,000 policies. In comparison, validation takes less time, averaging 8 to 10 microseconds when the SGX enclave is not used. Although there is a small increase, we see that it is negligible compared to the overall performance. The large standard deviation of the result is due to the context switching that occurs during request validation within the enclave. In general, context switching occurs when the required policy data is not present in the cached memory page, resulting in a significant execution delay, especially in the SGX enclave, due to the encryption and decryption processes required during page replacement. In particular, retrieving policy information from memory requires decryption of the encrypted memory, which contributes to the observed policy validation delay.

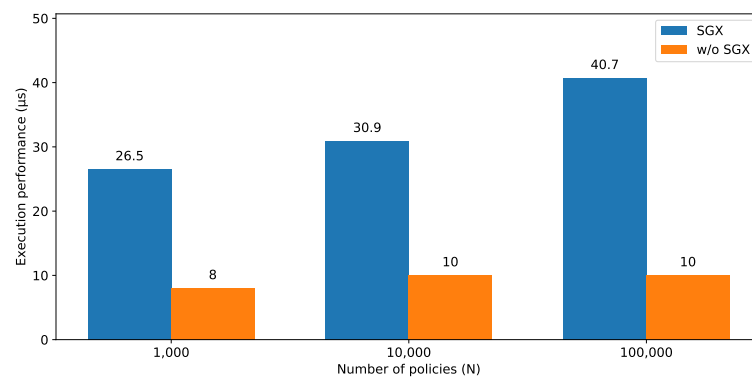


Figure 12. Execution performance according to the number of policies.

While SGX enclave validation causes a slight increase in processing time, especially under conditions involving context switches, this increase is in the microsecond range and does not exceed 1 millisecond even under the most demanding conditions. Thus, although SGX validation is slightly slower, the impact on overall system performance is minimal.

7.2. Scalability for Large-Scale Workload Deployment

Since the controller keeps a list of access control policies within an SGX enclave, the limited memory capacity of the SGX enclave restricts S-ZAC's ability to support a large number of workloads and policies. To overcome this limitation, S-ZAC has proposed an efficient data storage scheme designed for supporting large-scale deployments. Specifically, S-ZAC has introduced a data structure that includes 'index' and 'rule' fields as members of the structure.

To evaluate the scalability of S-ZAC, we conducted an experiment to figure out the maximum number of workloads and access control policies it can support. In the implementation for the structure, the 'index' field is a 32-byte character array, and the 'rule' field is an 8-byte pointer used to dereference the policy rules. This results in a total size of 40 bytes for the structure.

Given that Intel SGX supports limited EPC sizes of less than 256MB, S-ZAC can deploy up to 6.7 million policies when a single workload is running. Figure 13 illustrates the number of access control policies that S-ZAC can support according to the number of workloads in the cloud. As expected, the number of policies decreases as the number of workloads running in the cloud increases. These results indicate that if the administrator deploys 50 access control policies per workload, S-ZAC can support approximately 134,217 workloads in the cloud.

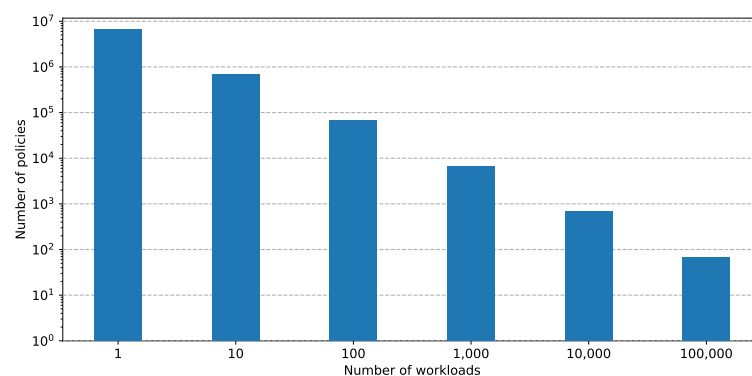


Figure 13. The number of workloads that can be supported according to the number of policies configured for each workload.

7.3. Micro-Benchmarking

To analyze the quantitative factor of SGX runtime overhead that affects the overall performance of S-ZAC, we perform micro-benchmarking on two S-ZAC SGX-shielded components, controller and deployer. Each S-ZAC component has a separate process context, under which multiple threads run within an SGX enclave. In this experiment, we measure the execution time of each running thread and compare the result with the case where the SGX enclave is not used in S-ZAC.

Table 3 shows the micro-benchmarking results. The results are obtained by conducting 10 independent experiments and averaging these measurements. We observe that the SGX runtime overhead affects the execution time of threads from 1.6 times to 102 times. We attribute one of the performance-influencing factors to the usage of RA-TLS to establish trusted communication between S-ZAC components. The RA-TLS provisioning process consists of multiple steps, including the generation of certificates and private keys through the issuance and signing of SGX quotes via PCCS. Most steps of the RA-TLS provisioning are omitted in the SGX-disabled case where normal TLS certificates are used.

Table 3. Micro-benchmarking result.

Process	Thread	Time (w/ SGX)	Time (w/o SGX)
Controller	Policy Executor	4.1 ms	0.04 ms
	Session Manager	2.49 ms	0.02 ms
	Session Listener	1.41 sec	0.64 ms
	Policy Receiver	1.41 sec	0.83 ms
Deployer	Policy Distributor	1.42 sec	0.14 ms
	Policy Manager	1.8 ms	0.05 ms

8. Security Analysis and Discussion

We analyze the security of S-ZAC in terms of the security requirement outlined in Section 4.2.

8.1. Security Analysis

Defending against a threat T1. S-ZAC relies on SGX for control plane confidentiality and integrity, which are the necessary security properties to achieve the security objectives against a threat T1. All memory content, including code and data, used by SGX enclaves is protected with cryptographic algorithms. To accomplish this, SGX has built-in robust memory encryption and memory isolation techniques that use encryption of the physical enclave memory. The decryption keys are stored in a hardware-level MEE, preventing access even with root privileges. As a result, the control plane of S-ZAC, which resides in an enclave, can ensure confidentiality and integrity using SGX features.

In the following, we provide more concrete arguments about the security of S-ZAC against the threat T1 (cf. Section 4.2.2).

Mitigating A1. As the physical memory that SGX enclaves use is encrypted, any attempts to read content inside the enclaves (e.g., using memory dump tools) will only reveal ciphertexts to an attacker. Without appropriate decryption keys, the attacker cannot obtain any confidential information about the control plane from the ciphertexts. Certainly, this security feature is effective in preventing adversaries from leaking any confidential data residing in the enclave memory.

Mitigating A2. The Intel SGX memory encryption engine uses a MAC to ensure the integrity of the encrypted enclave contents. MAC verification detects any attempts to modify or corrupt the encrypted data. Thus, without the correct MAC keys, an attacker cannot manipulate the data and code of an SGX-shielded control plane.

Mitigating A3. S-ZAC generates any secrets, such as a TLS certificate and the corresponding private key, related to secure communication or configuration through an SGX remote attestation. Thus, the generated certificate and a private key, as well as enclave

measurements, are being kept inside the SGX enclave protection boundary. Attempts to leak them through memory leak attacks are thwarted.

Defending against a threat T2. In our design, two critical S-ZAC control plane components, a controller and a deployer, are isolated inside SGX enclaves. As these components are protected by SGX technology, it is impossible for the attacker to obtain information about control plane processes. Thus, S-ZAC ensures that the attacker cannot impact the control plane's operation in any way. Besides, S-ZAC relies on remote attestation to authenticate the legitimacy of the control plane. Successful remote attestation ends in generating master secrets used for RA-TLS. S-ZAC ensures the secure management of the secrets as well as private keys for TLS certificates by isolating them inside the SGX enclaves.

In the following, we provide more concrete arguments about the security of S-ZAC against the threat T2 (cf. Section 4.2.2).

Mitigating A4. As S-ZAC mounts all control plane processes within SGX enclaves, it can protect the control plane process from attackers. Even with host OS-level privileges, an adversary cannot terminate processes within the enclave. Consequently, it becomes challenging for any adversary to impact the S-ZAC control plane.

Mitigating A5. Through SGX attestation, S-ZAC verifies the integrity of S-ZAC component executables before they are loaded into the SGX enclave. Any unauthorized attempts by attackers to corrupt the executables are detected and blocked.

Mitigating A6. To perform an MITM attack, the attacker needs to establish sessions with both a controller and a deployer. However, without proper RA-TLS private keys, which are concealed within SGX enclaves, the attacker cannot impersonate either the controller or the deployer. Thus, it is infeasible for the attacker to deliver MITM attacks.

Mitigating A7. An MITM attack conducted on the master node aims to create sessions with both the master and worker nodes' control planes, allowing the attacker to manipulate network communication between them. However, within the S-ZAC environment, performing such attacks requires establishing sessions with the deployer and controller and obtaining their private keys, which is impossible.

8.2. Discussion

While Intel SGX provides a robust, trusted execution environment for securing application code, there are some challenges associated with using the SGX enclave. In this section, we discuss several potential limitations of S-ZAC with respect to SGX usage.

Requiring trust in remote attestation. The primary principle of zero trust is to avoid granting implicit trust to every component within the system. However, S-ZAC requires a trusted node for the PCCS, which is a crucial component necessary for utilizing Intel SGX DCAP without Internet access. The necessity of trust in the PCCS arises from keeping certificates of the deployer and the controller for remote attestation. This deployment practice violates the fundamental principles of the zero trust paradigm.

To address this limitation, we can restrict access to PCCS exclusively through the state-of-the-art SGX library supported by Intel [53]. Utilizing the SGX library to interact with PCCS enforces the use of a PCCS API key, which is essential for authenticating workloads prior to accessing PCCS. Moreover, as communication with PCCS is conducted over HTTPS, the workload has the capability to authenticate itself to PCCS.

Cloud service providers also allow third parties to deploy PCCS in their environments. For example, IBM Cloud provides a guideline for users to deploy a virtual private cloud (VPC) in their own environment for remote attestation using DCAP [54]. This enables S-ZAC to establish a zero-trust architecture without requiring trust for the cloud service provider.

Mitigating SGX attacks. Recent studies have revealed that SGX is subject to various attacks [43–46], which can affect the security of S-ZAC. For instance, rollback [55] and replay attacks [56] are among possible security threats against the sealing technology that is responsible for maintaining the persistent state of SGX. In design, S-ZAC avoids using the

vulnerable SGX sealing technology, as presented in detail in Section 5.2. Therefore, S-ZAC is not vulnerable to these SGX attacks.

9. Conclusions

In this paper, we proposed S-ZAC, an access control hardening technique for the service mesh-based solution in the cloud. S-ZAC aims to improve the security of the service mesh's control plane, particularly in response to the two threats from privileged attackers: (1) circumventing access control checks; and (2) disturbing access control enforcement. To achieve this, S-ZAC enforces access control by isolating access control functions within an SGX enclave and ensuring its integrity whenever it performs policy deployment and access control enforcement. With these approaches, the trustworthiness of the control plane can be ensured even in the presence of privileged attackers. In addition, S-ZAC leverages Intel SGX remote attestation to ensure integrity and achieve zero trust. We address three issues that can arise when utilizing S-ZAC: a secure communication channel, scalability issues as workloads grow, and resiliency for the recovery of failure. We evaluated the effectiveness of S-ZAC in terms of performance overhead in enforcing access control and provisioning with a proof-of-concept implementation of S-ZAC. The experimental results show that S-ZAC outperforms other service mesh solutions such as Istio in terms of response time with a negligible request loss rate. In addition, the result shows that the policy validation overhead does not affect the overall system performance, with the performance impact remaining in the microsecond range. However, S-ZAC has several potential limitations. It requires a trusted node for the PCCS, and due to the limited memory capacity of the SGX enclave, it cannot handle large-scale workload deployments. Therefore, we plan to address these challenges in the near future.

Author Contributions: Conceptualization, C.H.; methodology, C.H. and T.K.; software, C.H. and W.L.; validation and visualization, C.H. and W.L.; writing—original draft, C.H. and T.K.; writing—review and editing, T.K. and Y.S.; supervision Y.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by a National Research Foundation of Korea (NRF) grant, funded by the Korean government (MSIT) (No.2023R1A2C2006862).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Stafford, V. Zero trust architecture. *NIST Spec. Publ.* **2020**, *800*, 207.
- Chandramouli, R.; Butcher, Z. A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments. *NIST Spec. Publ.* **2023**, *800*, 207A.
- Rodigari, S.; O'Shea, D.; McCarthy, P.; McCarry, M.; McSweeney, S. Performance analysis of zero-trust multi-cloud. In Proceedings of the 2021 IEEE 14th International Conference on Cloud Computing (CLOUD), Chicago, IL, USA, 5–11 September 2021; pp. 730–732.
- Sedghpour, M.R.S.; Townend, P. Service mesh and ebpf-powered microservices: A survey and future directions. In Proceedings of the 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE), Newark, CA, USA, 15–18 August 2022; pp. 176–184.
- Dzogovic, B.; Santos, B.; Hassan, I.; Feng, B.; Jacot, N.; Van Do, T. Zero-Trust cybersecurity approach for dynamic 5g network slicing with network service mesh and segment-routing over IPv6. In Proceedings of the 2022 International Conference on Development and Application Systems (DAS), Suceava, Romania, 26–28 May 2022; pp. 105–114.
- CVE-2019-5736. Available from MITRE, CVE-ID CVE-2019-5736. 2019. Available online: <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-5736> (accessed on 11 August 2024).
- CVE-2020-1527. Available from MITRE, CVE-ID CVE-2020-1527. 2020. Available online: <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-1527> (accessed on 11 August 2024).
- Machado de Sousa, E.; Shahzad, A. Data Loss Prevention from a Malicious Insider. *J. Comput. Inf. Syst.* **2022**, *62*, 1101–1111. [CrossRef]
- Choudhary, A.; Bhadada, R. Insider Threat Detection and Cloud Computing. In *Advances in Data and Information Sciences: Proceedings of ICDIS 2021*; Springer: Singapore, 2022; pp. 81–90.

10. Rizvi, S.; Williams, I. Analyzing Transparency and Malicious Insiders Prevention for Cloud Computing Environment. *Comput. Secur.* **2023**, *137*, 103622. [CrossRef]
11. Costan, V.; Devadas, S. Intel SGX Explained. Cryptology ePrint Archive, Paper 2016/086. 2016. Available online: <https://eprint.iacr.org/2016/086> (accessed on 11 August 2024).
12. Niemi, A.; Pop, V.A.B.; Ekberg, J.E. Trusted Sockets Layer: A TLS 1.3 based trusted channel protocol. In Proceedings of the Nordic Conference on Secure IT Systems, Virtual, 29–30 November 2021; Springer: Cham, Switzerland, 2021; pp. 175–191.
13. Bailleu, M.; Thalheim, J.; Bhatotia, P.; Fetzer, C.; Honda, M.; Vaswani, K. Speicher: Securing lsmbased key-value stores using shielded execution. In Proceedings of the 17th USENIX Conference on File and Storage Technologies (FAST 19), Boston, MA, USA, 25–28 February 2019; pp. 173–190.
14. Kim, T.; Park, J.; Woo, J.; Jeon, S.; Huh, J. Shieldstore: Shielded in-memory key-value storage with sgx. In Proceedings of the Fourteenth EuroSys Conference 2019, Dresden, Germany, 25–28 March 2019; pp. 1–15.
15. Alder, F.; Kurnikov, A.; Paverd, A.; Asokan, N. Migrating SGX enclaves with persistent state. In Proceedings of the 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Luxembourg, 25–28 June 2018; pp. 195–206.
16. Jangid, M.K.; Chen, G.; Zhang, Y.; Lin, Z. Towards formal verification of state continuity for enclave programs. In Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), Virtual, 11–13 August 2021; pp. 573–590.
17. Tsai, C.C.; Porter, D.E.; Vij, M. Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX. In Proceedings of the 2017 USENIX Annual Technical Conference (USENIX ATC 17), Santa Clara, CA, USA, 10–11 July 2017; pp. 645–658.
18. Scarlata, V.; Johnson, S.; Beaney, J.; Zmijewski, P. *Supporting Third Party Attestation for Intel SGX with Intel Data Center Attestation Primitives*; White Paper; 2018; p. 12. Available online: <https://www.intel.com/content/dam/develop/external/us/en/documents/intel-sgx-support-for-third-party-attestation-801017.pdf> (accessed on 11 August 2024).
19. Corporation, I. Intel Trust Authority. 2023. Available online: <https://www.intel.com/content/www/us/en/security/trust-authority.html> (accessed on 11 August 2024).
20. Adam, C.; Adebayo, A.; Franke, H.; Snible, E.; Feldman-Fitzthum, T.; Cadden, J.; Jean-Louis, N. Partially Trusting the Service Mesh Control Plane. *arXiv* **2022**, arXiv:2210.12610.
21. Zhang, L.; Li, H.; Ge, J.; Wu, Y.; Li, L.; Wu, B.; Deng, H. EDP: An eBPF-based Dynamic Perimeter for SDP in Data Center. In Proceedings of the 2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS), Takamatsu, Japan, 28–30 September 2022; pp. 1–6.
22. Isovalent, I. eBPF-Based Networking, Observability, Security, 2014. Available online: <https://cilium.io/> (accessed on 11 August 2024).
23. Duong, V.B.; Kim, Y. A Design of Service Mesh Based 5G Core Network Using Cilium. In Proceedings of the 2023 International Conference on Information Networking (ICOIN), Bangkok, Thailand, 11–14 January 2023; pp. 25–28.
24. Hussain, F.; Li, W.; Noye, B.; Sharieh, S.; Ferworn, A. Intelligent service mesh framework for api security and management. In Proceedings of the 2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), Vancouver, BC, Canada, 17–19 October 2019; pp. 735–742.
25. Kang, M.; Shin, J.S.; Kim, J. Protected coordination of service mesh for container-based 3-tier service traffic. In Proceedings of the 2019 International Conference on Information Networking (ICOIN), Kuala Lumpur, Malaysia, 9–11 January 2019; pp. 427–429.
26. Yang, C.; Tan, L.; Shi, N.; Xu, B.; Cao, Y.; Yu, K. AuthPrivacyChain: A blockchain-based access control framework with privacy protection in cloud. *IEEE Access* **2020**, *8*, 70604–70615. [CrossRef]
27. Gupta, R.; Kanungo, P.; Dagdee, N.; Madhu, G.; Sahoo, K.S.; Jhanjhi, N.; Masud, M.; Almalki, N.S.; AlZain, M.A. Secured and privacy-preserving multi-authority access control system for cloud-based healthcare data sharing. *Sensors* **2023**, *23*, 2617. [CrossRef] [PubMed]
28. Saini, A.; Zhu, Q.; Singh, N.; Xiang, Y.; Gao, L.; Zhang, Y. A smart-contract-based access control framework for cloud smart healthcare system. *IEEE Internet Things J.* **2020**, *8*, 5914–5925. [CrossRef]
29. Messadi, I.; Neumann, S.; Weichbrodt, N.; Almstedt, L.; Mahhouk, M.; Kapitza, R. Precursor: A fast, client-centric and trusted key-value store using rdma and intel sgx. In Proceedings of the 22nd International Middleware Conference, Québec City, QC, Canada, 6–10 December 2021; pp. 1–13.
30. Priebe, C.; Vaswani, K.; Costa, M. EnclaveDB: A secure database using SGX. In Proceedings of the 2018 IEEE Symposium on Security and Privacy (S&P), Francisco, CA, USA, 21–23 May 2018; pp. 264–278.
31. Yang, Z.; Li, J.; Lee, P.P. Secure and Lightweight Deduplicated Storage via Shielded Deduplication-Before-Encryption. In Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC 22), Carlsbad, CA, USA, 11–13 July 2022; pp. 37–52.
32. Pires, R.; Pasin, M.; Felber, P.; Fetzer, C. Secure content-based routing using intel software guard extensions. In Proceedings of the 17th International Middleware Conference, Trento, Italy, 12–16 December 2016; pp. 1–10.
33. Nakatsuka, Y.; Paverd, A.; Tsudik, G. PDoT: Private DNS-over-TLS with TEE support. *Digit. Threat. Res. Pract.* **2021**, *2*, 1–22. [CrossRef]
34. Schwarz, F.; Rossow, C. SENG, the SGX-Enforcing Network Gateway: Authorizing Communication from Shielded Clients. In Proceedings of the 29th USENIX Security Symposium (USENIX Security 20), Boston, MA, USA, 12–14 August 2020; pp. 753–770.
35. Nakano, T.; Kourai, K. Secure offloading of intrusion detection systems from VMs with Intel SGX. In Proceedings of the 2021 IEEE 14th International Conference on Cloud Computing (CLOUD), Chicago, IL, USA, 5–11 September 2021; pp. 297–303.

36. Kim, S.; Han, J.; Ha, J.; Kim, T.; Han, D. Enhancing security and privacy of tor's ecosystem by using trusted execution environments. In Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17), Boston, MA, USA, 27–29 March 2017; pp. 145–161.
37. Li, W.; Lemieux, Y.; Gao, J.; Zhao, Z.; Han, Y. Service mesh: Challenges, state of the art, and future research opportunities. In Proceedings of the 2019 IEEE International Conference on Service-Oriented System Engineering (SOSE), San Francisco East Bay, CA, USA, 4–9 April 2019; pp. 122–1225.
38. Chandramouli, R.; Butcher, Z. Building secure microservices-based applications using service-mesh architecture. *NIST Spec. Publ.* **2020**, *800*, 204A.
39. Pan, X.; Bacha, A.; Rudolph, S.; Zhou, L.; Zhang, Y.; Teodorescu, R. Nvcool: When non-volatile caches meet cold boot attacks. In Proceedings of the 2018 IEEE 36th International Conference on Computer Design (ICCD), Orlando, FL, USA, 7–10 October 2018; pp. 439–448.
40. Gueron, S. A Memory Encryption Engine Suitable for General Purpose Processors. Cryptology ePrint Archive, Paper 2016/204, 2016. Available online: <https://eprint.iacr.org/2016/204> (accessed on 11 August 2024).
41. Johnson, S.; Scarlata, V.; Rozas, C.; Brickell, E.; Mckeen, F. Intel software guard extensions: EPID provisioning and attestation services. *White Pap.* **2016**, *1*, 119.
42. Knauth, T.; Steiner, M.; Chakrabarti, S.; Lei, L.; Xing, C.; Vij, M. Integrating remote attestation with transport layer security. *arXiv* **2018**, arXiv:1801.05863.
43. Nguyen, T.; Thai, M.T. Denial-of-service vulnerability of hash-based transaction sharding: Attack and countermeasure. *IEEE Trans. Comput.* **2022**, *72*, 641–652. [CrossRef]
44. Van Bulck, J.; Minkin, M.; Weisse, O.; Genkin, D.; Kasikci, B.; Piessens, F.; Silberstein, M.; Wenisch, T.F.; Yarom, Y.; Strackx, R. Foreshadow: Extracting the keys to the intel SGX kingdom with transient Out-of-Order execution. In Proceedings of the 27th USENIX Security Symposium (USENIX Security 18), Baltimore, MD, USA, 15–17 August 2018; pp. 991–1008.
45. Moghimi, D.; Van Bulck, J.; Heninger, N.; Piessens, F.; Sunar, B. CopyCat: Controlled Instruction-Level Attacks on Enclaves. In Proceedings of the 29th USENIX Security Symposium (USENIX Security 20), Boston, MA, USA, 12–14 August 2020; pp. 469–486.
46. Lipp, M.; Kogler, A.; Oswald, D.; Schwarz, M.; Easdon, C.; Canella, C.; Gruss, D. PLATYPUS: Software-based power side-channel attacks on x86. In Proceedings of the 2021 IEEE Symposium on Security and Privacy (S&P), San Francisco, CA, USA, 24–27 May 2021; pp. 355–371.
47. Kim, Y.; Daly, R.; Kim, J.; Fallin, C.; Lee, J.H.; Lee, D.; Wilkerson, C.; Lai, K.; Mutlu, O. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. *ACM SIGARCH Comput. Archit. News* **2014**, *42*, 361–372. [CrossRef]
48. Chen, Z.; Vasilakis, G.; Murdock, K.; Dean, E.; Oswald, D.; Garcia, F.D. VoltPillager: Hardware-based fault injection attacks against Intel SGX Enclaves using the SVID voltage scaling interface. In Proceedings of the 30th USENIX Security Symposium (USENIX Security 21), Vancouver, BC, Canada, 11–13 August 2021; pp. 699–716.
49. Intel. Introducing to Intel SGX Sealing, 2024. Available online: <https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-intel-sgx-sealing.html> (accessed on 11 August 2024).
50. Fei, S.; Yan, Z.; Ding, W.; Xie, H. Security vulnerabilities of SGX and countermeasures: A survey. *ACM Comput. Surv. (CSUR)* **2021**, *54*, 1–36. [CrossRef]
51. Hashicorp. Identity-Based Networking with Consul, 2023. Available online: <https://www.consul.io/> (accessed on 11 August 2024).
52. Foundation (CNCF). C.N.C. Simplify Observability, Traffic Management, Security, and Policy with the Leading Service Mesh. Available online: <https://istio.io/> (accessed on 11 August 2024).
53. Intel Corporation. Design Guide for Intel® SGX Provisioning Certificate Caching Service, 2020. Available online: https://download.01.org/intel-sgx/sgx-dcap/1.10/linux/docs/SGX_DCAP_Caching_Service_Design_Guide.pdf (accessed on 11 August 2024).
54. INM Cloud. Attestation with Intel SGX and Data Center Attestation Primitives (DCAP) for Virtual Servers for VPC, 2024. Available online: <https://cloud.ibm.com/docs/vpc?topic=vpc-about-attestation-sgx-dcap-vpc> (accessed on 11 August 2024).
55. Strackx, R.; Piessens, F. Ariadne: A Minimal Approach to State Continuity. In Proceedings of the 25th USENIX Security Symposium (USENIX Security 16), Austin, TX, USA, 10–12 August 2016; pp. 875–892.
56. Skarlatos, D.; Yan, M.; Gopireddy, B.; Sprabery, R.; Torrellas, J.; Fletcher, C.W. Microscope: Enabling microarchitectural replay attacks. In Proceedings of the 46th International Symposium on Computer Architecture, Phoenix, AZ, USA, 22–26 June 2019; pp. 318–331.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.